

SAFE 예상 질문 Q&A

발표 대비 꼬리질문 대응 자료

Q1. 왜 SQLite에서 MySQL로 바꿨나요?

초기에는 빠른 프로토타입 구현을 위해 SQLite를 사용했습니다.
데이터·테이블이 늘고 여러 페이지에서 같은 데이터를 조회하게 되면서
통합 관리와 확장성을 고려해 MySQL로 전환했습니다.

 SQLite로도 충분하지 않나요?

현재 규모에서는 가능하지만, 향후 확장까지 고려해 서버형 DBMS를 선택했습니다.

 바꾸면서 뭐가 좋아졌나요?

여러 분석 데이터를 테이블 단위로 통합 관리, SQLAlchemy + PyMySQL 로 연결·적재 방식
도 공통화했습니다.

Q2. 왜 Wide Format을 Long Format으로 바꿨나요?

원본 데이터는 연도가 각각 컬럼으로 구성된 경우가 많았습니다.

DB에서는 연도를 하나의 컬럼으로 관리하는 것이 **조회·집계·확장에 유리**해 pandas `melt()` 로 Long Format 변환했습니다.

Long Format의 장점은?

연도 추가 시 컬럼이 아닌 **행만 추가**하면 됨. `WHERE year=2025`, `GROUP BY year` **조회·집계 용이**

Wide Format은 나쁜 구조인가요?

아닙니다. 사람이 한눈에 비교할 땐 Wide가 편함. **SAFE는 저장·분석은 Long**, 필요 시 `pivot()` 으로 재변환

Q3. 전처리에서 가장 어려웠던 부분은?

기관마다 데이터 구조가 서로 달랐던 점입니다.

- Excel 병합 셀 · 다중 헤더 · CSV 인코딩 · 합계행 · 컬럼명 상이

→ 최종적으로 **지역·연도·연령·시간대** 공통 기준으로 표준화

병합 셀 처리는?

`ffill()` 로 상위 값 복원

CSV 한글 깨짐은?

`utf-8-sig` / `cp949` / `utf-8` / `euc-kr` 순차 확인·처리

Q4. 왜 pandas를 사용했나요?

수집 데이터 대부분이 Excel·CSV 정형 데이터였고,
결측치 처리·타입 변환·집계·`melt()`·`pivot_table()` 작업이 많이 필요했습니다.
MySQL·Streamlit과의 연동도 편리해 선택했습니다.

 데이터가 훨씬 많아져도 pandas를 쓸 건가요?

현재 규모는 충분. 증가 시 DB 사전 집계로 최적화하고, 더 커지면 Spark 등 분산 처리도 검토

Q5. 왜 사고건수 외에 사망자·부상자를 추가했나요?

사고건수만으로는 사고의 심각성을 설명하기 어렵다고 판단해
TAAS 원본에서 사고건수 + 사망자 + 부상자를 함께 전처리해 DB에 추가했습니다.

 어떤 분석이 가능해졌나요?

사고 100건당 사망자, 사고 1건당 부상자 등 파생지표로
사고 규모를 보정한 심각도 분석이 가능해짐

Q6. 왜 Streamlit을 사용했나요?

핵심은 복잡한 프론트엔드 구현이 아니라 **분석 결과를 빠르게 웹에서 제공하는** 것이었습니다. Python 기반이라 pandas·Plotly 결과를 바로 화면에 연결할 수 있어 선택했습니다.

React는 왜 안 썼나요?

React는 UI 자유도·대규모 확장에 강점이 있지만, 이번 프로젝트는 **데이터 처리·분석이 핵심**이라

제한된 개발 기간에는 Streamlit이 더 적합했습니다.

Q7. 왜 ARIMA나 LSTM을 사용하지 않았나요?

데이터가 2021~2025년, 짧은 연 단위 시계열이라 복잡한 모델을 학습·검증하기엔 관측치가 부족했습니다.

현재 데이터에서 해석 가능한 Linear Trend + Proxy Growth Scenario를 선택했습니다.

- 🔄 ARIMA란? 과거 값·과거 예측 오차의 관계를 이용하는 통계적 시계열 모델

- 🔄 LSTM이란? 이전 시점 정보를 기억하는 순환신경망 기반 딥러닝 모델 — 많은 학습 데이터 필요

- 🔄 Prophet은 왜 안 썼나요? 장기 일·월 데이터의 계절성 분석에 강점이 있으나, SAFE는 연 단위·짧은 기간이라 장점 활용 어려움

Q7-1. (이어서) 향후에는 어떻게 개선할 건가요?

과거·신규 연도 데이터를 지속적으로 추가한 뒤
ARIMA·Prophet 등을 적용하고

MAE · RMSE

등의 평가 지표로 모델 성능을 비교할 계획입니다.

Q8. Linear Trend가 정확히 무엇인가요?

과거 데이터의 **증가·감소 흐름을 직선 형태로 모델링**하고,
그 추세가 이어진다고 가정해 미래 값을 계산하는 방식입니다.

갑자기 추세가 바뀌면요?

실제값과 차이가 커질 수 있음. 그래서 확정값이 아니라
"현재 추세가 지속된다는 가정하의 전망치"로 제공합니다.

Q9. Proxy Growth Scenario는 무엇인가요?

직접 예측할 데이터가 부족할 때, 관련성 높은 다른 지표의 증가율을 **대리변수(Proxy)**로 활용해 미래 변화 시나리오를 계산하는 방식입니다.

 Proxy와 실제 지표가 같이 움직인다는 보장이 있나요?

보장할 수 없습니다. 그래서 정확한 예측이 아니라

"관련 지표 변화가 지속된다는 가정하의 탐색 시나리오"로 활용했습니다.

Q10. 미래 예측값을 신뢰할 수 있나요?

정확한 미래 사고건수를 단정하는 값이 아니라,
현재 추세가 지속된다는 가정하의 **변화 방향·규모를 살펴보는 전망치**입니다.
정책 결정의 절대 기준이 아니라 **위험 탐색용 참고지표**입니다.

정확하지 않은데 의미가 있나요?

어떤 지역·지표가 상대적으로 빠르게 증가할지 확인해 **정책 검토가 필요한 영역을 선제 탐색**할 수 있습니다.

정확도는 어떻게 검증하나요?

데이터가 충분히 쌓이면 Train/Test 분리 후 MAE·RMSE로 모델별 성능 검증 예정

Q11. 데이터는 어떻게 갱신하나요?

신규 원본 데이터 → 전처리 → 기존 테이블 TRUNCATE
→ 최신 데이터 재적재 → Streamlit 조회

 전체 재적재는 비효율적이지 않나요?

맞습니다. 현재는 **정합성·구현 단순성**을 우선해 전체 재적재 방식을 사용했고,
규모가 커지면 **증분 적재** 방식으로 개선할 계획입니다.

 자동 갱신은 가능한가요?

Open API 제공 데이터는 향후 API 수집 → 전처리 → DB 적재 자동화 계획입니다.

Q12. 왜 Plotly를 사용했나요?

사용자가 Hover 값 확인 · 확대/축소 · 범례 선택 등 상호작용을 할 수 있도록 사용했고, Streamlit과의 연동도 편리했습니다.

Matplotlib은 왜 안 썼나요?

정적 분석 그래프에는 충분하지만, 웹 대시보드에서는 Plotly의 인터랙티브 기능이 사용자 경험 측면에서 더 적합했습니다.

Q13. 왜 여러 종류의 그래프를 사용했나요?

다양하게 만드는 것 자체가 목적이 아니라 **분석 목적·데이터 특성에 맞게 선택했습니다.**

분석 목적	시각화
시간에 따른 변화	Line Chart
지역·연령 규모 비교	Bar Chart
월·시간대 집중도	Heatmap
지역적 차이	Choropleth Map
핵심 수치	KPI Metric

 **선택 기준은?** 사용자가 무엇을 비교해야 하는가를 기준으로 선택

Q14. 정책 대응 우선순위는 어떻게 정했나요?

단순히 현재 사고건수가 높은 지역만 보지 않고

현재 위험 수준 + 미래 변화 가능성을 함께 확인해 정책 검토 필요 지역을 탐색하도록 구성했습니다.

 1위 지역이 가장 위험하다는 뜻인가요?

아닙니다. SAFE 데이터·가정에 따른 **상대적 우선순위**이며,
실제 정책 결정에는 교통환경 등 **추가 요인**을 함께 고려해야 합니다.

Q15. 가장 중요하게 해결한 기술적 문제는?

여러 공공기관의 서로 다른 데이터 구조를 하나의 서비스로 표준화한 것입니다.

(Excel 구조·병합 셀·헤더·인코딩·연도 표현 방식 등)

→ 지역·연도·연령·시간대 공통 기준으로 통합

구체적 사례 하나만요?

TAAS 사고 데이터: 처음엔 사고건수만 적재 → 심각도 분석 필요성 확인

→ 원본 재확인 → 사망자·부상자 전처리 추가 → DB 스키마 수정 → 재적재 → 화면 반영

Q16. 본인이 담당한 부분은 어디까지인가요?

데이터 수집 이후

전처리 구조 설계 → DB 구축·적재 → 스키마 수정 → Streamlit 구현 → Plotly 시각화 → 미래
전망까지

데이터가 최종 화면에 표현되는 **전체 기술 흐름**을 담당했습니다.

가장 자신 있는 부분은?

전처리부터 DB·분석 화면까지 **전체 파이프라인을 연결한 부분.**

새 지표가 필요하면 화면만이 아니라 원본 데이터·전처리·DB 스키마까지 함께 수정해 반영했습니다.



발표 직전 핵심 6문장

1. SQLite → MySQL: 데이터·테이블 증가에 따른 통합 관리·확장성을 위해 전환
2. Wide → Long: 연도를 컬럼으로 관리해 SQL 조회·확장을 쉽게 하기 위해 변환
3. 데이터 전처리: 기관마다 다른 구조를 지역·연도·연령 등 공통 기준으로 표준화
4. 예측 모델: 짧은 시계열에 적합하고 해석 가능한 방법(Linear Trend/Proxy) 선택
5. 예측 신뢰성: 확정값이 아닌, 추세 지속 가정하의 전망·정책 참고지표
6. 향후 개선: 데이터 축적 후 ARIMA·Prophet 비교, 증분 적재·API 자동화로 고도화